# Gemma Reads the Decision. Does the Code Follow?

Oscar Mörke | Independent researcher | 8 November 2026

## Abstract

In a local coding pilot, Gemma read a named project-decision file in every instructed episode, yet working feature completion remained rare. We separate three outcomes that a continuity evaluation can otherwise conflate: accessing evidence, selecting the current decision, and producing a valid software artifact. A Gemma 4 E2B pilot covered six scenarios of one coding feature, two conditions and three repetitions. File reads were 18/18 with a pointer and read instruction and 0/18 without; functional completion was 1/18 in each condition. The decision grader's two passes did not establish application of recovered evidence. A separate configuration study covered twelve synthetic tasks and 108 episodes. A complete handover and a versioned broader packet supplied the same latest answer at the same cutoff. Both achieved 35/36 primary successes, versus 5/36 when the request omitted the desired value. Broader context missed the preset improvement threshold over handover. This contrast concerns supplied information and package form; it cannot establish retrieval benefit, equivalence, or improved coding. A third, prospectively frozen study then asked the coding question directly: twelve runnable Python tasks, 108 completed episodes, hidden tests for policy, function and unrelated behavior. With the decision supplied, Gemma produced a fully passing patch in 11/36 (handover) and 12/36 (evidence) episodes, versus 3/36 without it. Five of twelve tasks were never solved in any arm. Supplying the decision helped, but in most episodes the decided behavior never reached the code; of the 30 patches that did implement it, 7 failed other tests. The contribution is a bounded evaluation case: reading, decision correctness and working code need separate checks, because each one can pass while the next fails.

## 1. The question: can a recovered decision become working code?

A coding assistant can find the right note and still leave a broken feature. In our pilot, even a passing decision test could reward rejecting valid configurations. A read trace alone therefore offered little assurance that the requested software change worked.

We ask a narrow question: what do local tests of session continuity actually establish about a coding agent? We distinguish evidence access, latest-decision correctness and artifact validity. The pilot observes the first directly and exposes weaknesses in the other measures. Expansion A then uses a simpler artifact and a stricter composite grader. Its equal-information handover control asks whether broader supplied history adds value beyond a complete concise account. Study B returns to runnable code with a design frozen before any scored call, so the original question gets a direct answer.

These are separate studies with different tasks and interfaces. Their scores must not be combined into a before-and-after improvement claim. No learned memory method, retrieval architecture or general benchmark is introduced. The practical insight is an evaluation failure mode: a system can satisfy an intermediate check while failing the software task that motivated it.

## 2. Gemma coding pilot: access succeeded, application unresolved

### Design and execution

The 2 November pilot tested a Click CLI configuration feature under six decision scenarios. Each scenario received pointer and no-pointer conditions, repeated three times: 36 episodes. The pointer condition named the decision file and instructed a read. Thus its contrast combines location information with an explicit instruction; it does not isolate either component.

The saved model identity is Gemma 4 E2B QAT Q4_0, served locally through llama.cpp. Context was 8192 tokens, with six calls and a total generation budget of 8192 tokens per episode. Protocol and tests were hashed before scoring. The task family is one feature under different decisions, rather than six independent repository problems. Saved outcomes and audit records are identified as P1 in Section 7.

### Outcomes and grader failure

| Pilot outcome | Pointer + read instruction | No pointer |
|---|---|---|
| Decision file read | 18/18 | 0/18 |
| Functional completion | 1/18 | 1/18 |
| Original decision-test pass | 1/18 | 1/18 |

The read threshold passed; the decision-benefit threshold did not. The decision-test passes require qualification. In one pointer episode the implementation rejected all configurations, including valid cases. The no-pointer pass arose through generic behavior without reading the record. Neither demonstrates successful application of recovered evidence. We preserve the original scores and disclose the false positives rather than silently replacing the historical grader.

Functional completion was 2/36 overall. The saved harness audit reports eleven import failures, five replies cut by the length limit and eight episodes with unexecuted writes at the call limit. These diagnostics are not a partition of the episodes. The termination categories, which do partition them, were fourteen malformed replies, thirteen call-limit endings, six done endings and three budget endings. The parser treated bare done replies as malformed. These interface failures and the functional floor prevent a clean estimate of decision benefit. Planned human adjudication of all transcripts remains undone.

## 3. Expansion A: matched answers in supplied context

### Task, interventions and controls

The 3 November study uses twelve synthetic JSON configuration tasks. Each fresh repository begins with the same settings.json and README, without decision records. The request names a key and omits its desired value. Request-only receives no additional evidence. Handover receives a concise account of the latest decision. Broader receives a versioned decision, session excerpt and later correction. Both supplied packages contain the same latest answer at the same cutoff and are exactly 160 whitespace-separated words; they are not tokenizer-matched. Six tasks contain a stale conflict, and source order varies.

This design distinguishes answer availability from a comparison between informative package forms. The request-only arm lacks information needed to determine the intended value. Its contrast with supplied context therefore cannot identify a special memory mechanism. The broader packet is supplied manually; no retriever is evaluated. The matched handover is the relevant control for an incremental history claim.

The same quantized Gemma file ran with 8192-token context, serial execution and two host threads. Each episode allowed six tool calls, 8192 generated tokens and a 600-second wall limit. Tools constrained writes to settings.json; model code was not executed. Twelve tasks, three conditions and three repetitions yielded 108 scored episodes. Task/repetition blocks were shuffled and condition position balanced. The source, schedule, graders and reporting rules were frozen after disjoint practice and before scored execution. Primary records are E1-E3.

Primary success requires the latest target value together with valid JSON, the original key set, string types and unchanged unrelated settings. Duplicate keys fail. Functional validity checks those invariants and a valid target enum without requiring the latest value. This distinction prevents a syntactically acceptable wrong choice from counting as decision success. Frozen controls include valid reference configurations and failures for wrong choices, collateral edits, malformed or empty artifacts and reject-all behavior. The historical control record is

preserved; this revision does not claim a fresh model experiment.

### Results and interpretation

| Supplied context | Latest + valid artifact | Functional validity | Latest-source citation |
|---|---|---|---|
| Request only | 5/36 | 15/36 | N/A |
| Complete handover | 35/36 | 36/36 | 35/36 |
| Broader packet | 35/36 | 36/36 | 1/36 |

The primary totals tie at 35/36 for both informative arms. All twelve task totals tie, although the failed repetitions differ on T11. With stale conflicts, each informative arm achieves 17/18; without conflicts each achieves 18/18. Those subsets contain six constructed tasks each and offer limited evidence about history burden.

Rule D1 required broader to exceed request-only by at least 9/36; the observed margin was 30/36, so it passed. Rule D2 required a margin of at least 6/36 over handover and no worse task totals in at least nine of twelve tasks. Its margin was zero, so it failed. These were descriptive operational thresholds, not significance or equivalence tests. Equal totals at this ceiling do not establish equivalence or absence of an effect.

Handovers explicitly reminded the model which source ID to cite; broader packets lacked that reminder. The citation contrast is consequently confounded and does not establish weaker evidence use by the broader arm. Request-only has no citation denominator. Citation and behavior are separate measures.

A disk-full interruption stopped the supervisor after 63 completed episodes. Inspection preceded resumption of unfinished entries; completed episodes were retained without reruns. The unfinished attempt lacks a saved transcript, though startup was retained. Saved scored episodes account for 202,048 generated tokens and 3471.713 episode-seconds. The server version was captured during execution, not in the freeze.

This study changes the artifact, parser and context intervention relative to the pilot. Its higher success rate cannot demonstrate better coding or explain the pilot's failures. It shows that the tested model often applied an explicitly supplied value to a small constrained artifact.

## 4. Study B: can a supplied decision become a working patch?

### Design, frozen before scoring

Study B asks the pilot's question again with runnable code. Twelve new synthetic Python tasks cover four API transitions, four configuration migrations and four regression repairs. Each task has a base repository, a reference patch and hidden tests in three groups: policy (the decided behavior), functional (the feature works) and collateral (unrelated behavior is unchanged). Primary success requires all three groups, the exact file structure and untouched immutable files. Grading reads final files only, never explanations or citations.

Three arms each ran three repetitions: 108 planned episodes. Request receives the request and repository, with the policy facts absent. Handover adds a complete handover at a fixed cutoff. Evidence adds the latest decision content at the same cutoff. The two informative arms carry byte-identical policy facts and differ only in their wrapper label. Request is therefore an answer-absent diagnostic. The handover versus evidence contrast tests wrapper form, not retrieval or memory.

Before any scored call, all twelve reference patches passed and all 84 negative controls failed: base, wrong policy, empty source, reject-all, unrelated regression, immutable-file change and extra test file for every task. An independent model review (Composer 2.5) read the source of all twelve tasks and found no blocker. An earlier focused challenge found that the grader rejected attribute names starting with one leading "_" and allowed only a narrow builtin set, while the prompt did not say so. The frozen version states both rules in the system prompt shared by every arm. The review covered source only; it is not a model or human validation.

The runtime matched the earlier studies: the same Gemma 4 E2B QAT Q4_0 file, served by llama.cpp 0.8.0, build 11960, with a 16384-token context. Each episode allowed six replies, 2048 tokens per reply, 8192 generated tokens in total and 600 seconds. Sampling used temperature 0.7 and top-p 0.95. Episodes ran serially in an order shuffled with a fixed seed. A fresh unscored qualifier case passed before scoring began.

### Results

All 108 episodes completed inference. Primary records are S1-S3.

| Arm | Full pass | Policy | Functional | Collateral |
|---|---|---|---|---|
| Request (no decision) | 3/36 | 3/36 | 4/36 | 26/36 |
| Handover | 11/36 | 16/36 | 17/36 | 31/36 |
| Evidence | 12/36 | 14/36 | 18/36 | 35/36 |

| Family (12 episodes per cell) | Request | Handover | Evidence |
|---|---|---|---|
| API transitions | 0 | 3 | 5 |
| Configuration migrations | 0 | 2 | 1 |
| Regression repairs | 3 | 6 | 6 |

All three request passes came from one regression task, R02, which all arms solved 3/3; its fix does not depend on the decision. Seven of twelve tasks were solved at least once by an informative arm. Five tasks (A03, A04, M02, M04, R03) were never solved in any arm. Per task, evidence minus handover was +2 on A01, +1 on M03, -2 on M01 and zero on the other nine. The protocol fixed no superiority threshold for this contrast, and a one-pass difference on twelve shared tasks supplies no measured benefit for either wrapper.

The policy column is the central result. Even with the decision supplied, the decided behavior appeared in code in only 16/36 (handover) and 14/36 (evidence) episodes. Of those 30 patches, 23 also passed every functional and collateral test; 7 broke something else. All 82 failing episodes kept the required file structure and immutable files; they failed on hidden behavior cases, not on format.

Episodes ended with done 68 times, at the reply limit 37 times and at the token limit 3 times. Of the 26 passes, 21 ended with done. Scored episodes used 434,594 generated tokens and 7,392 episode-seconds.

### Attempt history

Three earlier attempts of the same frozen study stopped on operational guards: two on low disk space after 48 and 12 episodes, and one after 5 episodes when a guard detected another desktop application. Their 65 episodes, with 9 full passes, are retained as separate incomplete cohorts and never pooled. Attempt 4 reran all 108 episodes from the start, with no retries inside the attempt. Only attempt 4 is reported above.

### Limits

These are small synthetic functions written by the same author as the graders, with finite hidden cases. The four migrations share one dictionary conversion structure. Repetitions share task units, so the task, not the episode, is the analysis unit. One model and one workstation were measured. A matched second model, independent task authors and human review of the 82 failures are the next steps.

## 5. Earlier private recall audit: methodological context

A 14 October workstation audit motivated this work: twenty private questions, four harness conditions, five repetitions, 400 scored outcomes. None of its four preset decision thresholds passed. Missing startup snapshots, possible permission confounds, unpinned CLI versions and imperfectly blinded model grading limit it to context. It is not a Gemma or matched multi-model result. Private questions stay withheld; H1 lets readers replay the arithmetic only.

## 6. Relation to software engineering and memory evaluation

SWE-bench evaluates edits against repository issues [1]. SWE-agent studies how an agent's interface affects its ability to navigate, edit and test software [2]. Those settings motivate our insistence on working artifacts and interface diagnostics; our configuration tasks provide much narrower evidence. LongMemEval separately evaluates information extraction, reasoning, updates and abstention in conversational histories [3]. Our separation of evidence access and artifact success addresses a related measurement concern in a local software setting, without claiming comparable coverage.

The software-engineering implication is a proposed evaluation practice: report evidence access, decision correctness and regression-preserving completion separately, then require their appropriate conjunction for a claim about applying project history. The pilot's false positives show why the grader must accept valid behavior as well as reject invalid behavior. Expansion A demonstrates such a conjunction only for constrained JSON edits. Study B applies it to runnable patches, where the decided behavior was the most frequent missing piece. Transfer to other models remains untested.

## 7. Evidence and reproducibility boundary

Companion research records remain private and are not distributed with this paper. The table identifies the saved evidence supporting each study; it is an inventory, not a download list.

| ID | Supports |
|---|---|
| P1 | Pilot reads, grading outcomes, budgets and failure diagnostics |
| E1 | All expansion totals, task pairs, conflicts and resource totals |
| E2 | Design, grading rules, package construction and recorded controls |
| E3 | Freeze identity, model hash and runtime caveats |
| H1 | Historical arithmetic and preset decision outcomes |
| S1 | All Study B totals, family and task counts, statuses, tokens and time |
| S2 | Frozen design, reference and mutant controls, independent source review |
| S3 | Runtime pins, qualifier, attempt history |

The earlier replay recomputed saved scores without new model calls. Study B records include a replay script that rebuilds every Study B number from the raw episode rows.

Arithmetic reproducibility is conditional on saved annotations. It does not validate private labels, reconstruct missing transcripts, replace independent human adjudication or promise identical model reruns. The studies use one workstation, one Gemma snapshot and limited task diversity. Study B tasks and graders share one author. Repetitions share task content, so episode counts cannot be treated as independent task sample sizes. We report descriptive contrasts without a pooled episode-level confidence interval.

## 8. Conclusion

Gemma reliably accessed an explicitly named decision file in this pilot, while functional completion remained low and the decision grader admitted false positives. On a separate constrained task, both informative context packages applied the latest supplied value in 35/36 episodes. Broader context failed its preset improvement threshold over a complete handover. On runnable patches, the supplied decision raised fully passing patches from 3/36 to 11/36 and 12/36, yet most episodes still failed hidden tests. The supported lesson is to check the full path from evidence to working artifact, preserve failed controls and bound conclusions to the task actually measured. For this small local model, the main loss lies between holding the decision and expressing it in code; most patches that did express it also kept the code working.

## References

[1] C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? 2023. https://arxiv.org/abs/2310.06770

[2] J. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. 2024. https://arxiv.org/abs/2405.15793

[3] D. Wu et al. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. 2024. https://arxiv.org/abs/2410.10813